

Week1 Introduction & Representing Text

Representing Text: Create representations that preserve the meaning, syntax, and semantics of text. Methods include word embeddings, phrase embeddings, and sentence embeddings.

Good Representation: Morphology, Animacy, Number, Semantic similarity.

Basic Word Representation: 1. Dictionary-Based: Represent words using definitions from dictionaries. 2. WordNet: Represents words through their relationships(Synonym sets, Hypernyms, Hyponyms, Meronyms)

Numeric Representations: 1. **One-hot Vectors**: [0, 0, ..., 1, ..., 0]. Dsadv: High-dimensional and sparse. Cannot capture similarity between words. 2. **Sparse Embeddings**: Use TF-IDF to represent words. Adv: Saves space. Disadv: Less efficient if many words are active. 3. **Map (dictionary)** (3, 2, 4; 1): Adv. Space-efficient; Easy to update, flexible. In when calculating similarity, you can only traverse the smaller dictionary, then use O(1) to find the corresponding frequency in the other dictionary. The overall complexity is about O(min(n, n)) (most efficient).

TF-IDF: Introduces weighting to improve BoW by downweighting common words and upweighting rare words. Use TF-IDF to replace word counts. **Benefit:** Easy to implement. Each feature corresponds to a human-readable reason for interpretability.

Limits: Can only see counts, not meanings; fails on subtle linguistic cues. Model becomes huge with big vocab; Ignores word order. No semantic relationship.

Distributional Semantics(Vicary): Idea: How to represent word through word co-occurrence matrices. Create large co-occurrence matrices from corpora. Apply dimensionality reduction (e.g., SVD) to convert into dense vectors. 2. **word2vec**: Make up a task and learn vectors with it. 3. **Continuous Bag of Words from word2vec**: "Continuous" because the embeddings have a floating point value. BoW: It inputs multiple context words as a bag, regardless of their order, but only considering their content. Characteristic: Similar words (+2 words), more syntactically similar words. Larger context (+/-5 words), more semantically similar words.

Evaluation of Word Representations: 1. **Dot Product**: longer vectors tend to have larger dot products. Not a pure measure of similarity, as it's influenced by magnitude rather than direction. 2. **Cosine Similarity**: Measures directional similarity, independent of vector magnitude. Range from -1to1. **Intrinsic Evaluation Metrics:** 1. **Word Analogy Task**: "father : mother :: doctor : nurse". Evaluate whether word embeddings capture analogical reasoning. **Caveats of Analogy Tasks:** Works better for frequent words. Captures only certain types of relations. Depends on the linear structure of embedding space. Word embeddings trained on real-world corpora can learn social biases.

Instability of Word Embeddings: Even when the dataset, model, training method, and inference method are fixed, changing only the random seed can lead to different embeddings. High-frequency words tend to have more stable embeddings. Variation suggests that the embedding space is not unique—many equivalent solutions exist.

Week 2 Foundation of NLP Systems

Core Components of NLP Systems: 1. **Data**: the form of input-output pairs.

Tokensation: Simple whitespace-based tokensisation works for English. 2. **Model Input**: A model needs a function f(v) that takes an input-output pair (v, y) and encapsulates how the system "thinks" about the problem. 3. **Inference Method**: Given an input, choose the best output using the method. 4. **Learning Method**: Update model parameters based on prediction errors. Perceptron Algorithm: If prediction is wrong: Add weights for the correct label with wrong sample. Subtract weights for the incorrect label with wrong sample. If true don't do any thing. 5. **Metrics**: Evaluate model performance. Loss: Cross-Entropy Loss (for SVM), Squared Error. Classification Metrics: Acc, Precision, Recall, F1 Score.

The Baseline Model: Feature: Simple architecture, Easy to implement and interpret, Fast to train and test, Performance benchmark

Word Vectors: are techniques that map discrete words into continuous vector space representations.

Dimensional word representations: Aggregate word vectors by averaging or weighted averaging.

Evaluation: To quantitatively compare model predictions against true labels. To guide model tuning and improvements.

ROC Curve: Plots True Positive Rate (left-TP/TP+FN) vs False Positive Rate (down-TP/FP+FN). AUC (Area Under Curve): Higher is better (maximum = 1, perfect classifier). When the distribution of positive and negative samples in the data changes, the ROC curve shifts. High AUC indicates good performance, where the data distribution of each category is relatively uniform, and can more comprehensively reflect the overall performance of the model. F1 evaluates the effectiveness of your current classification decision, and AUC evaluates the best performance your model can achieve under different decision strategies.

Precision, Recall, F1 (per class) – snapshot at the chosen threshold, good for comparing models.

ROC/AUC – holistic look at score distributions, critical for imbalanced or cost-sensitive deployments. **PRC Curve:** Plots left-precision vs down-recall. See if the curve is close to the upper right corner (indicating that the positive prediction is more accurate). PRC performs better in cases of unbalanced data and can more accurately reflect the performance of the model in a minority category.

3 Models – Non-linear

Neural Networks: The fundamental concept: introduce non-linear functions into models to increase their capacity to learn complex mappings.

Issues of Sigmoid Function

1. **Sigmoids saturate and kill gradients.** A undesirable property of the sigmoid unit is that its saturating output prevents gradients from flowing backward through it. The gradient at these regions is almost zero. (same to tanh)

2. Sigmoid outputs are not zero-centered.

Hyperbolic Tangent Function (Tanh)

The main advantage provided by the function is that it produces zero-centred output thereby aiding the back-propagation. Also vanishing gradient issues.

ReLU Function: When the input is positive, the gradients don't vanish. This helps to mitigate the vanishing gradient problem.

The main advantage of using the ReLU in computation is that, they guarantee faster computation. Another property of the ReLU is that it introduces sparsity in the hidden units as it squashes the values between zero to maximum.

Multi-Layer Perceptron

Solves the limitations of a single-layer perceptron by introducing: 1. Multiple hidden layers. Each added layer composes a non-linear activation, letting deep nets express functions that would require exponentially many units in a shallower linear-equivalent network.

Universal Approximation Theorem

Equivalent to the **Universality Theorem**, which states that "Neural networks with a single hidden layer can be used to approximate any continuous function with arbitrary precision".

MLP can identify any continuous non-linear patterns.

Without activation MLP: All matrix products of linear maps remain linear, and the bias terms can be concatenated into one overall offset.

Backpropagation

use backpropagation to compute gradients: Represent forward computation through a simple neural graph, then compute the gradient of loss with respect to every weight and bias. Back-propagation applies the chain rule by sequentially multiplying each layer's local derivative with the gradient flowing from the layer above, thereby converting the loss's derivative with respect to the network's output into derivatives with respect to every weight.

Regularization

Motivation: Avoid Overfitting, Improve Generalization, Feature Selection

L1: Lasso Regression (Weight Penalty)

By reducing the size of the weights, the model is less likely to fit the noise in the data and is more likely to generalize well to new data.

L2: Shrinks weights towards zero, discouraging the model from learning extreme, highly specialised parameters; this pushes the network towards simpler decision boundaries that generalise better.

Effect: push learning to favour smaller weights (L2) or more zero weights (L1).

Stop Stopping: When validation error not improved for some time, stop.

Dropout

At training (each iteration): Each unit is retained with a probability p. At test: The network is used as a whole. The weights are scaled-down by a factor of p.

Forces the network to rely on multiple, redundant pathways rather than any single fragile pattern, effectively training an ensemble of sub-nets whose average performs better than any one.

Dropout has more advantages over L1/L2

is scale-free: dropout does not penalize the use of large weights when needed.

is invariant to parameter scaling: dropout is unaffected if weights in a certain layer are scaled up by a constant c, and weights in another layer are scaled down.

Recurrent Models (RNNs)

Motivation: The Usage of Memory

Sequential Dependency: The current output often depends not just on the current input, but also on the preceding inputs.

An RNN uses its memory to maintain this context. Handle sequences of any Length. allows it to capture these dynamics.

Neural Network + Memory = RNN

Initial hidden state: 1. **Set to zero.** The advantage of this is that it is simple to implement, and the zero vector can often be used as a "default" initialization.

2. **Learnable initial state.** This allows the model to adjust its initial memory or "bias" during training.

Issues of RNN

Vanishing Gradient Issue: The derivatives of some activation functions (sigmoid, tanh) become very small when the input value is large or small. This causes the gradients of these functions to decay layer by layer during the back propagation process. Consequence: model "forgets" long-term dependencies.

• **Exploding Gradients.** In an RNN, error gradients can accumulate during an update and result in very large gradients. Solution: Gradient clipping—cap the gradient value.

• **The problem of learning long-range dependencies.** RNN may not be able to capture the information of the previous information if the distance is too far.

Long Short-Term Memory (LSTM)

Motivation: To solve the problem of learning long-term dependencies in sequence data.

Methodology

Forget gate: model decides what information from the long-term state C_{t-1} should be thrown away or kept

Input gate layer: Decides what new information should be stored in cell state

Update the cell state: Update the old cell state C_{t-1} to produce the new state C_t by combining the values of the forget and input layers

Output gate layer: Decides what exactly to output - the output will be based on C_t and a tanh activation function

Forget gate: will be a filtered version of C_{t-1} and a previous hidden state h_{t-1}

Input gate: will be a filtered version of C_t and a previous hidden state h_{t-1}

Output gate: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Hidden state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Cell state: will be a filtered version of C_t and a previous hidden state h_{t-1}

Evaluation Criteria: To count as a true positive (TP), the start, end, and label must all match between prediction and ground truth. Span appears only in the start and end labels. False Positive: Span appears only in ground truth = False Negative.

True negatives (TN) are not usually computed due to the large number of possible spans. During evaluation, if we count all unselected intervals as "True Negative", the number will be very large (because most intervals are not entities)

Why NER Use Micro Metrics?

In the NER task, some labels may be relatively common, while others may appear less frequently. Micro-average will calculate all categories of instances together. If macro-average is used, the indicators of each category are calculated independently first, and then the category indicators are averaged.

This will give the minority class the same weight as the majority class.

Week 4 Inference – Greedy and Search

Exhaustive Search

The most straightforward idea is to **enumerate all possible candidates** and select the one with the highest score. Time complexity: $O(2^{|V|})$

Method: Iterate through all possible outputs. Compute score for each input-output pair. Return the y with the highest score

Benefits: It provides the gold/optimal standard for output prediction. It can be used for any structured prediction task where the output space is enumerable.

Limitations: Even small input sizes can lead to huge search spaces.

Greedy Search

Exhaustive search becomes infeasible in such cases. Greedy Search offers an efficient alternative by making locally optimal decisions at each step. Time complexity for linear: $O(n * |tags|)$

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

Limitations: Greedy decisions can lead to locally optimal but globally sub-optimal sequences. Once a choice is made, it cannot be revised, even if it leads to later errors. Greedy decoding always outputs the same sequence for the same input and often repeats the label that has been output.

Beam Search

Method: Treat sequence prediction as a step-by-step decision-making process, choosing the best local option at each time step.

Benefits: No need to maintain paths, candidate sets, or complex data structures.

2. Use directed arcs to connect a head (parent) word to its dependent child.

3. Find a dependency tree (or graph) that satisfies certain constraints (acyclicity, etc.).

Scoring Model: Consider a sentence $w_1, w_2, \dots, w_n$. Suppose we define a score function $s(w_i \rightarrow w_j)$ for every possible arc. ($w_i \rightarrow w_j$)

Constraints

Acyclic/Produce a tree: There must be no cycles in the set of arcs; otherwise, the structure is not a valid tree.

No Crossing Arcs: Many standard settings, arcs are required to be projective (i.e., they do not cross when drawn over a linear word order).

Single Root: There is one special node "ROOT", and every other token should have exactly one parent.

The tree has the highest score of all possible trees.

Backtracking

Once the DP table is fully computed, we retrieve the best score from dp[0][n][empty type], then backtrack through the stored "best splits" and "best arcs" to reconstruct the final dependency tree.

Coreference refers to identifying whether multiple mentions in a text point to the same real-world entity.

Method: Step 1: identifying all possible entity mentions in the text. Step 2: For each newly appearing mention m_i, we look at all preceding mentions m_j (where j<i) and decide if they refer to the same entity. Step 3: After pairwise linking. We then apply transitive closure so that any mentions connected directly or indirectly end up in the same entity cluster.

Limits: If a sentence has length n, the number of possible mentions can be up to O(n²), and all possible clusterings is exponential.

Week 6 Models – Encoder-Decoder

Encoder

Motivation: Early NLP models often used one-hot representations or word frequency methods (like TF-IDF), which have high dimensionality and cannot directly capture semantic or similarity relationships between words. Goal: Represent words as dense, low-dimensional vectors where semantically similar words are close in space.

Contextual Bag of Words (CBOW): Use context words to predict the target word. BoW: The context word vectors are typically averaged or summed before feeding into the network, do not care about the order.

Advantages: Tends to perform well on shorter text segments because it uses all surrounding words to predict the center word. Often trains faster than Skip-Gram in many cases.

Drawbacks: When the window is large, simple averaging might dilute critical words' influence. On certain sparse contexts or longer sequences, Skip-Gram can outperform it.

Skip-Gram

Reverse of CBOW: use "center word" to predict its surrounding context words.

Advantages: On large datasets, it often yields better performance because each target word has to predict multiple surrounding words. It handles rare words better. The model's similarity between words is used to predict its context.

Drawbacks: Involves more computation (each center word predicts multiple context words). In practice, negative sampling or hierarchical softmax is employed to speed up training.

Negative Sampling

Computing a full softmax over thousands of words is costly. Negative sampling involves computing softmax by focusing on the true context words and only a small number of negative examples (sampled non-context words) at each step.

Benefit for word2vec

The word vectors generated by Word2Vec distribute similar words closer in the embedding space. It can also reflect certain analogy relationships (e.g., "king" - "man" + "woman" ≈ "queen").

Limits for word2vec: word2vec is sensitive to word order. Word2Vec assigns a single vector per word, failing to model polysemy. If a word never appears in the training set, the model cannot produce any embedding for it. Even if the window size is increased, it does not dynamically change embeddings based on context.

GloVe

Word2Vec only use "local context" information and have relatively limited use of global statistical information. Some global statistical methods (such as using co-occurrence matrices) can capture the global co-occurrence of words well, but the vector space may be too sparse.

GloVe aims to combine global co-occurrence information with low-dimensional vector distributed representation to form word vectors (the dot product of two vectors is approximately the co-occurrence value).

Limits: GloVe involves encoding word-word co-occurrences from the entire corpus, avoiding the local-only limitation of purely window-based approaches. Empirically, GloVe vectors often show interpretable relationships, e.g., classical analogies such as "king + woman ≈ queen".

Limits: Like word2vec, GloVe yields one fixed vector per word, regardless of context. Constructing and storing the co-occurrence matrix X for a large corpus can be memory-intensive. GloVe does not offer a built-in approach to address out-of-vocabulary words.

fastText

Motivation: Addressing the OOV Issue: Traditional word embedding models rely on a fixed vocabulary. Traditional models treat each form as an independent token, leading to data sparsity and large vocabularies. fastText tackles this by using subword units (character-level) modelling, thus reducing dependence on a fixed vocabulary.

The core idea of fastText is to decompose a word into several character-level n-grams and then aggregate these n-gram embeddings to form the final word vector. The training objective is similar to word2vec (specifically Skip-Gram).

Benefit: Given that any new word can be decomposed into known character n-grams. By working at the subword level, fastText effectively models the similarities among different words, even if they share the same root, leading to closer overall representations for morphologically related words.

Limits: The vector for a given subword combination is static in any context—meaning it does not address the issue of polysemy (multiple senses for one word). The approach does not factor in sentence-level structure or higher-level semantics like dependency or semantic roles.

ELMo

Motivation: Before ELMo, the word vectors commonly (such as word2vec, GloVe, fastText) were all "static", that is, the vector representation of the same word in all sentences remained unchanged.

the vocabulary reaches the desired size or no further merges are possible.

WordPiece

Similarities and Differences Compared to BPE

Similar to BPE in that it learns subword units based on data-driven segmentation. Difference: WordPiece prefers to use language model likelihood or probability-based metrics to select subword pairs when merging subwords.

Merging Criterion

Each potential merge (forming a new subword) is evaluated in terms of its effect on the language model perplexity. The merge that yields the greatest improvement (largest drop in perplexity) is selected.

HuggingFace WordPiece

HuggingFace, when re-implementing WordPiece, simplified and optimised **how merging decisions are made**. They use a very straightforward scoring method:

```
score = [first symbol] × [second symbol]
[combined]
- [first symbol]: frequency of the first subword
- [second symbol]: frequency of the second subword
Intuitive understanding: This formula favours merging pairs that appear frequently together but are not too frequent individually.
```

Unigram / SentencePiece

Stands out with a large set of possible subword units (ranging from single characters to entire words). Iteratively removes subwords that make the smallest contribution to overall likelihood, until the vocabulary shrinks to the desired size. **Key Characteristics:** It allows splitting at any position in the text, particularly useful for CJK (Chinese, Japanese, Korea) scripts or languages that do not rely on whitespace.

Conclusion: Tokenization

BPE word segmentation is fragmented and does not consider language structure, relying only on character frequency.

Unigram LM word segmentation can retain meaningful subword units, which is more in line with the morphology and semantics of natural language.

Therefore, in complex tasks such as language modeling and machine translation, Unigram LM often works better.

Feed-Forward Network (FFN) **Lowercase and normalize tokens:** "Apple", "APPLE", and "apple" are the same word semantically; treating them as one token reduce the vocabulary size. Normalising punctuation prevents the model from learning false distinctions that do not carry meaning.

Attention

Addressing the Information Bottleneck Problem: In RNN-based Seq2Seq, the model compresses the entire input sequence into a single fixed-length vector, which was degraded performance on long sentences because too much information had to be squeezed into one vector. The **core motivation** of the attention mechanism is to allow decoder to dynamically focus on and retrieve relevant parts of the input sequence instead of relying solely on one vector.

Method: at each decoding step t , compute a set of attention scores over the encoder hidden states $h_{1:T}$ then via softmax, and use them to produce a weighted sum of encoder hidden states.

Benefit: Alleviating the Bottleneck Problem; Handling Long-Range Dependencies: Attention allows "direct lookup" across the entire input, making it easier to capture relationships between distant words

Limits: Attention appears to be aligned but is not causally related in essence. For long sequences, attention mechanisms require $O(N^2)$ time and memory.

Attention Variants

Dot Product Attention

Simple and fast to compute. Only works when s and h have the same dimension. However, when the vector dimension is large, the dot product can become very large, causing softmax to produce extremely sharp distributions and leading to unstable gradients.

Scaled Dot-Product Attention

When the vector dimension is large, the dot product will increase, which can easily saturate the softmax (most probabilities are close to 0), affecting training. The term $\sqrt{d_k}$ is a scaling factor to prevent large dot products from shrinking the gradients too much after the softmax. Dividing by $\sqrt{d_k}$ can stabilize the training process and improve training efficiency.

Multiplicative Attention / Bilinear Attention **Project s and h into a lower-dimensional space using U and V respectively in parallel, and compute the attention score in that space.** Can improve efficiency by reducing computational and memory costs while preserves expressive power.

Additive Attention / Feedforward Attention

Non-linear, capable of capturing more complex interactions. Supports different dimensions between h and s. More computationally expensive than simple dot product but more expressive.

Reduced-rank Multiplicative Attention

Project s and h into a lower-dimensional space using U and V respectively in parallel, and compute the attention score in that space. Can improve efficiency by reducing computational and memory costs while preserves expressive power.

Additive Attention / Feedforward Attention Non-linear, capable of capturing more complex interactions. Supports different dimensions between h and s. More computationally expensive than simple dot product but more expressive.

Weak 7 Models – Transformer

Self-attention **Motivation:** Bottleneck issue; Limited parallelism: Each RNN step depends on the output of the previous step, making it hard to fully parallelize the computation over long sequences.

Method: Every token forms query–key–value dot-products with all other tokens, so information about distant words is combined in a single layer; no repeated computation over time steps.

Benefits: Self-attention operations for all positions within the same layer can be computed in parallel via matrix multiplications. The self-attention mechanism allows any two tokens to attend to each other directly, thereby capture global dependencies.

Limits: Quadratic Time Complexity $O(n^2)$: For a sequence of length n , self-attention requires computing pairwise cosine for all (i, j) . Must Add Positional Embeddings: Unlike an RNN, where recurrence inherently preserves order, self-attention requires an additional mechanism to keep track of positions. Softmax over thousands of tokens: Weights spread thin, making it harder for the model to focus sharply on the truly relevant context.

Positional Embeddings

Motivation: Self-attention treats the input as an unordered set; no representation of the tokens yields the same attention scores because queries, keys and values are multiplied without reference to position.

Learned Positional Embeddings

Core Idea: For each position i in the sequence, learn a vector p_i of the same dimensionality as d as the token embedding.

Pros: Highly expressive. The model can directly learn any positional feature that can be learned by the task.

Cons: Limited extrapolation. If you train on sequences up to length N , then positions beyond N at inference time may be undefined or poorly learned.

Sinusoidal Positional Encoding

This method uses sine and cosine functions to represent positions

Advantages: The encoding has the quality that tokens with the same relative distance in the sequence have the same representation in certain components.

Extrapolation: Even if the input length exceeds that observed in training, one can still compute sine and cosine values for larger pos. No extra weights to learn or store; computational efficiency.

Drawbacks: Non-trainable. It will lack the flexibility of learned embeddings.

Rotary Positional Embeddings

Use a rotation matrix to "rotate" the vector by a certain angle as the position changes; When calculating the query-key dot product, the "two words with a distance d " between them" can maintain similarity in certain components.

Transformer Encoder

Multi-Head Mechanias

The "multi-head" approach splits the embedding dimension into h subspaces, each performing attention independently, capturing different aspect. Finally, these h outputs are concatenated and transformed by a linear layer.

Position-Wise Feed-Forward

To alleviate vanishing gradients in deep networks and allow the model to preserve the original input. Make learning smoother.

Layer Normalization (LayerNorm)

LayerNorm computes mean and variance along the feature dimension for each sample vector. LayerNorm stabilizes training, especially in NLP tasks with variable sequence lengths or batch sizes.

Feed-Forward Network (FFN) **Add Nonlinear Transformation:** After the self-attention layer, each position's vector (the attention output) is fed into a fully connected feed-forward network

Position-Wise Independence: This feed-forward network applies independently to each token's representation; it does not mix information across positions but provides a richer feature transformation for each position. Transformer Encoder

Input and Positional Encoding

During training (teacher forcing). During inference, it takes the tokens generated so far in an auto-regressive manner.

Masked Self-Attention

Masking: To prevent a position i from "seeing" tokens in positions $j > i$, we use a mask matrix that sets the attention scores to $-\infty$ before the softmax.

Cross-Attention (Encoder-Decoder Attention) Query comes from the decoder hidden states, while Keys and Values come from the encoder hidden states.

Feed Forward Network (FFN): Similar to the encoder, each decoder layer includes a feed-forward sub-layer.

Residual Connections and Layer Normalization: Similar to the encoder

Output Layer (Softmax + Prediction) After the final decoder layer, a linear projection plus a softmax produce the probability distribution over the next token.

Scaling Laws

Methods to estimate the best model size given a certain amount of data and a budget for training. Research found that performance continues to improve as the model size, number of parameters, and training data size increase in a proportional fashion.

Training Complexity

FlashAttention focuses on an optimized implementation that reduces memory access overhead. Longformer lets each token attend only to a fixed-size window plus a few global tokens. **Feedforward Overhead:** In very large models, feedforward layers can become a significant computational bottleneck.

Weak 8 Models – Large Language Models **Language Models**

Definition: A language model assigns a probability to a sequence of words.

Big lookup table (Brute-Force Enumeration) If we could store the probability of every possible word combination ahead of time, we could retrieve the probability of any sequence in constant time.

Benefits: Given a sentence, you can directly retrieve the probability from the table without computation. Theoretically Most Accurate.

Limits: As the vocabulary size or sequence length increases $|V|^n$ becomes astronomically large. Data Sparsity Problem. No Generalization Ability.

Chain Rule

Using a lookup table for every possible word sequence is unrealistic. We need a systematic way to break down a complex problem into smaller, manageable parts to make computation feasible.

$$P(w_1, w_2, \dots, w_n) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times \dots \times P(w_n|w_1, w_2, \dots, w_{n-1})$$

Benefit: The chain rule breaks down a high-dimensional joint probability problem into a series of low-dimensional conditional probabilities. Language Generation

Limits: To fully estimate high-order conditional probabilities like $P(w_5|w_1, w_2, w_3, w_4)$, an enormous amount of training data is required. In real-world datasets, most specific long sequences never appear, making it hard to reliably estimate their conditional probabilities. Even after applying the chain rule, computing and storing all conditional probabilities can still be very costly.

Quantization: Reduces the precision of weights and activations to save memory and speed up computation. Post-training quantization can degrade model accuracy.

Benefits: Reduces memory usage by $\sim 4\times$. Increases inference speed, especially on hardware optimised for int8.

Limits: Quantization can degrade model accuracy. Generation tasks are sensitive to low precision.

QLoRA

Combine LoRA's fine-tuning efficiency with Quantization's compression benefits

Mixture of Experts (SMoE, etc.) Don't activate the entire model for every input. We dynamically activate only a few "experts" based on the input.

Benefit: Reduces computation while maintaining a very large total capacity.

Limits: Routing design is complex.

Mamba: Scales linearly with sequence length $O(N)$ instead of $O(N^2)$. Captures long-range dependencies efficiently. Enables parallel training.

Jamba

Combines the strengths of both Transformers and State Space Models. Seeks to leverage both for better scalability and performance across diverse tasks.

Weak 9 Training – Unsupervised In-Context Learning (ICL)

Rather than modifying weights of the model, provide instructions and examples in the input and hope that it will somehow use those to model the task.

0-shot prompting means giving the model no examples – only a task instruction is provided to guide the model.

1-shot prompting means providing one example before asking the model to perform a new instance of the task.

Few-shot prompting means providing multiple examples.

Chain-of-Thought prompting means requiring not just the final answer but also an explanation of the reasoning steps, guiding the model to think step-by-step.

Pros and Perplexity

Prompts with lower perplexity result in higher accuracy. Prompts with similar meanings but different wording yield different performance. A low-perplexity prompt starts with wording, structure and punctuation the model already knows well, so the conditional probabilities of the subsequent tokens are high.

Limits: Re-using very common phrases can amplify corpus biases and reduce diversity. Takeaway: Carefully chosen, representative, and diverse examples may be pruned away, hurting recall.

Controlled perplexity – Keep the first few words familiar but vary later context tokens to input the model to a wider range of outputs.

Prompt Variability

Different examples affect performance

Observation: Changing few-shot example set can cause accuracy to vary by over 20%. Takeaway: Carefully chosen, representative, and diverse examples improve generalisation.

Different prompt formats affect accuracy Findings: The syntax structure and description of prompts have a great impact on performance. Suggestions: Use clear, structured, and semantically clear

encoder, Train the model to decode the original text.

Masked Span Prediction: Randomly select a continuous span in the input sentence. The task is to predict the special tokens. The input to the model is the masked sentence; the target is to reconstruct the missing text.

Deleted Sequence Prediction: Delete parts of the input sentence directly, without using placeholders or mask tokens. The model must recover the missing parts based only on remaining context.

Deleted Span Prediction: Similar to Deleted Sequence Prediction, but instead of deleting parts of the sentence, the model predicts missing tokens.

Permututed Sequence Prediction: Randomly permute the order of tokens or phrases in the input sentence. The model reconstructs correct natural ordering.

Rotated Sequence Prediction: Rotate the input sentence by cutting a part from the sequence and moving it to the beginning or end. It maintains continuity but shifts the sentence starting point.

Infilling Sequence Prediction: Randomly blank out parts of input sentence and ask the model to fill in the missing content.

Pretraining: Training on large-scale, generic corpora to learn general language patterns. Characteristics: Performed once. Expensive and time-consuming. Requires multiple GPUs. Uses raw text as both input and output

Fine-tuning: Adaptation to a specific task using smaller, labelled datasets. Characteristics: Can be performed multiple times. Can be light or heavy depending on resources. Task-specific training

Evaluating LLMs

Mean Reciprocal Rank (MRR)

Aims to quantify how well the model predicts the next token, especially for retrieval-based tasks or classification ranking.

Limits: Only considers the position of first correct answer, ignoring the overall distribution of results. Mostly useful for retrieval tasks, less so for pure language modeling.

Perplexity

We need a method to measure how well the model predicts an entire sequence of text.

Benefit: The lower the perplexity, the better. (Lowest possible value is 1 when the probability is 1, No upper limit)

Limits: Highly sensitive to tokenisation strategy. ("N" depends on tokenisation) In real-world generation tasks, lower perplexity does not always correlate with better quality (e.g., coherence, creativity).

Directly tied to the cross-entropy loss used during training: $\text{Perplexity} = e^{\text{CrossEntropy}}$

Extrinsic Evaluation

BLEU score for translation. ROUGE score for summarisation. Word Error Rate (WER) for speech recognition.

Limits: These metrics measure specific model objectives. Fewer assumptions about data. Requires task-specific evaluation design.

Knowledge Distillation

Training steps:

1. The teacher model predicts soft target distributions. 2. The student model learns from the teacher's output rather than just the ground truth labels. 3. The student model is trained to match the teacher's output.

Benefits: Student models are much smaller. Faster inference, lower memory footprint. Reduced training cost.

Limits: Some complex abilities of the teacher may be lost. Requires a high-quality teacher model to start with.

Pruning

Iteratively repeat the above process multiple times: train → prune → reset → retrain. Each iteration increases the level of sparsity, while retraining helps the network recover accuracy. End up with a highly sparse network that retains similar accuracy to the original dense model.

Benefits: Significant reduction in model size.

Limits: Naive pruning can heavily hurt performance. The model structure and training data must be carefully chosen to maintain performance.

LoRA (Low-Rank Adaptation) Instead of full fine-tuning: Insert low-rank matrices into the model weights. Only train these small matrices while keeping the base model frozen.

Benefits: Greatly reduces memory usage during fine-tuning. Enables training large models even on smaller GPUs.

Limits: Only applicable to fine-tuning, not pre-training.

Quantization: Reduces the precision of weights and activations to save memory and speed up computation. Post-training quantization: reduce weight and activation precision to int8. Or Quantization-Aware Training: incorporate quantization during training.

Benefits: Reduces memory usage by $\sim 4\times$. Increases inference speed, especially on hardware optimised for int8.

Limits: Quantization can degrade model accuracy. Generation tasks are sensitive to low precision.

QLoRA

Combine LoRA's fine-tuning efficiency with Quantization's compression benefits

Mixture of Experts (SMoE, etc.) Don't activate the entire model for every input. We dynamically activate only a few "experts" based on the input.

Benefit: Reduces computation while maintaining a very large total capacity.

Limits: Routing design is complex.

Mamba: Scales linearly with sequence length $O(N)$ instead of $O(N^2)$. Captures long-range dependencies efficiently. Enables parallel training.

Jamba

Combines the strengths of both Transformers and State Space Models. Seeks to leverage both for better scalability and performance across diverse tasks.

Weak 9 Training – Unsupervised In-Context Learning (ICL)

Rather than modifying weights of the model, provide instructions and examples in the input and hope that it will somehow use those to model the task.

0-shot prompting means giving the model no examples – only a task instruction is provided to guide the model.

1-shot prompting means providing one example before asking the model to perform a new instance of the task.

Few-shot prompting means providing multiple examples.

Chain-of-Thought prompting means requiring not just the final answer but also an explanation of the reasoning steps, guiding the model to think step-by-step.

Pros and Perplexity

Prompts with lower perplexity result in higher accuracy. Prompts with similar meanings but different wording yield different performance. A low-perplexity prompt starts with wording, structure and punctuation the model already knows well, so the conditional probabilities of the subsequent tokens are high.

Limits: Re-using very common phrases can amplify corpus biases and reduce diversity. Takeaway: Carefully chosen, representative, and diverse examples may be pruned away, hurting recall.

Controlled perplexity – Keep the first few words familiar but vary later context tokens to input the model to a wider range of outputs.

Prompt Variability

Different examples affect performance

Observation: Changing few-shot example set can cause accuracy to vary by over 20%. Takeaway: Carefully chosen, representative, and diverse examples improve generalisation.

Different prompt formats affect accuracy Findings: The syntax structure and description of prompts have a great impact on performance. Suggestions: Use clear, structured, and semantically clear

prompt templates.

Ordering of examples influences prediction Findings: The order in which the category represented by the last example. This effect is particularly strong in early GPT models.

Number of In-Context Examples

Key Results: Accuracy improves rapidly within the first 2–4 examples, then plateaus. Larger models require fewer examples to achieve strong performance.

Scaling Up In-Context Examples

Finding: Natural/Random order leads to better accuracy. Grouping examples by type (e.g., Flipped labels) catch up with scale, suggesting that LLMs can learn label semantics from context.

Effects of Label Quality: Do label types matter?

Interpretation: Even wrongly labeled examples provide format or structure guidance to the model. Correct labeling is still ideal.

Label Semantics at Scale: Default vs. Abstract vs. Flipped labels

Finding: Default labels work best, especially in low-shot settings. Flipped labels catch up with scale, suggesting that LLMs can learn label semantics from context.

Chain-of-Thought Prompting: Reasoning improves performance Result: Instruction-tuned models benefit significantly from Chain-of-Thought. CoT improves reasoning-intensive tasks. CoT widens the model's effective context window for reasoning, mitigating "shortcut" answers and improving generalisation to multi-step tasks.

Interplay of Reasoning and Number of Shots

Result: Rationales and large numbers of examples reinforce each other. Structured reasoning improves data efficiency—you need fewer examples for the same performance.

Retrieval Augmentation

Retrieval Augmentation aims to enhance the capabilities of LLM by providing access to external information, addressing limitations: Limited internal knowledge of language models. Mitigating hallucination. Retrieving rare facts or up-to-date knowledge. Scaling knowledge without scaling model size.

Parallel Interaction

KNN-LM

For each generated token, use the current context vector to retrieve the nearest k context fragments and their next word from the training set context vector library, and the weight them by the nearest neighbor word distribution and the original distribution of the model.

Perplexity vs. Datascore Size

Adding the KNN module to a small model significantly reduces perplexity. The larger the datascore, the better the performance. A small model enhanced with a context database can outperform a much larger model trained from scratch.

Optimal k (interpolation parameter) vs. Datascore Size As the datascore increases, the quality of KNN predictions improves. Therefore, it becomes better to assign a higher weight to the KNN output.

Perplexity vs. k (number of nearest neighbors) Increasing k consistently reduces perplexity. However, the improvement diminishes logarithmically as k increases.

A vs. In-Domain and Domain Adaptation Perplexity For in-domain tasks, the original LM works well alone. Giving too much weight to KNN hurts performance. For domain adaptation, KNN helps significantly. Higher k leads to better performance.

Sequential Single Interaction

Retrieval-Augmented Language Model (RAG) Retriever: A dense vector retriever. Generator: A sequence-to-sequence model that generates text conditioned on the query and retrieved documents.

Retrieval ICL boosts performance significantly in few-shot scenarios without any training.

REALM (Retrieval-Augmented Language Model)

REALM is an end-to-end differentiable fashion of retriever and language model

RETRO (Retrieval-Enhanced Transformer) The generator applies cross-attention with the retrieved context.

Sequential Multiple Interaction FLARE (Flexible Language model-Augmented Retrieval Engine)

Key Takeaways: The model dynamically identifies knowledge gaps. It triggers task-specific retrievals instead of retrieving everything up front.

Weak 10 Data – Annotation and Crowdsourcing **IRC Dialogue Disentanglement Dataset**

Objective: Split the log into coherent sub-dialogues, where each is a complete, self-contained multi-turn conversation.

Characteristics: Very tight time spacing between messages (seconds apart). High interleaving of conversations. No explicit thread boundaries in raw data.

Limits:

GLUE is not a single task or dataset. It is a suite of diverse tasks designed to assess how well models perform across a broad range of semantic understanding challenges.

Risks in Dataset Construction

Shortcuts: Models might exploit superficial patterns or signals in data to give correct answers without actually understanding the task.

Validity issues: Answers are explicit in the text and often have only one possible correct value. This makes the questions trivially answerable by keyword matching rather than actual understanding or inference.

Avoiding Shortcuts and Enhancing Validity Use real-world data to avoid over-engineered biases. Create challenging examples (hard negatives).

Statistical power refers to a dataset's ability to support meaningful model comparisons. There are not enough difficult examples at the tail of the data, making it impossible to reliably evaluate the performance of the model.

Social Bias and